



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Anemone

使用Rust开发、多架构、多核、可动态扩展的宏内核

队员：张正翰、陈函申 指导老师：夏文、仇洁婷

全部决赛测例通过。截至八月十七日十三点整，排名和得分如下。

15	T202610486999873	我依旧是世一队	武汉大学	19	180.0	131.4	199.1	199.1	709.6000
16	T2026104869910397	CoreCraft	武汉大学	22	161.8	149.2	199.1	199.1	709.2000
17	T2026181239910972	斗世界电子开发部	哈尔滨工业大学 (深圳)	50	180.0	112.0	199.1	199.1	690.2000
18	T2026104229999620	比特工匠队	山东大学 (青岛)	9	138.3	123.7	199.1	199.1	660.2000
19	T2026100089999575	ShellCore	北京科技大学	27	109.7	123.7	199.1	199.1	631.6000

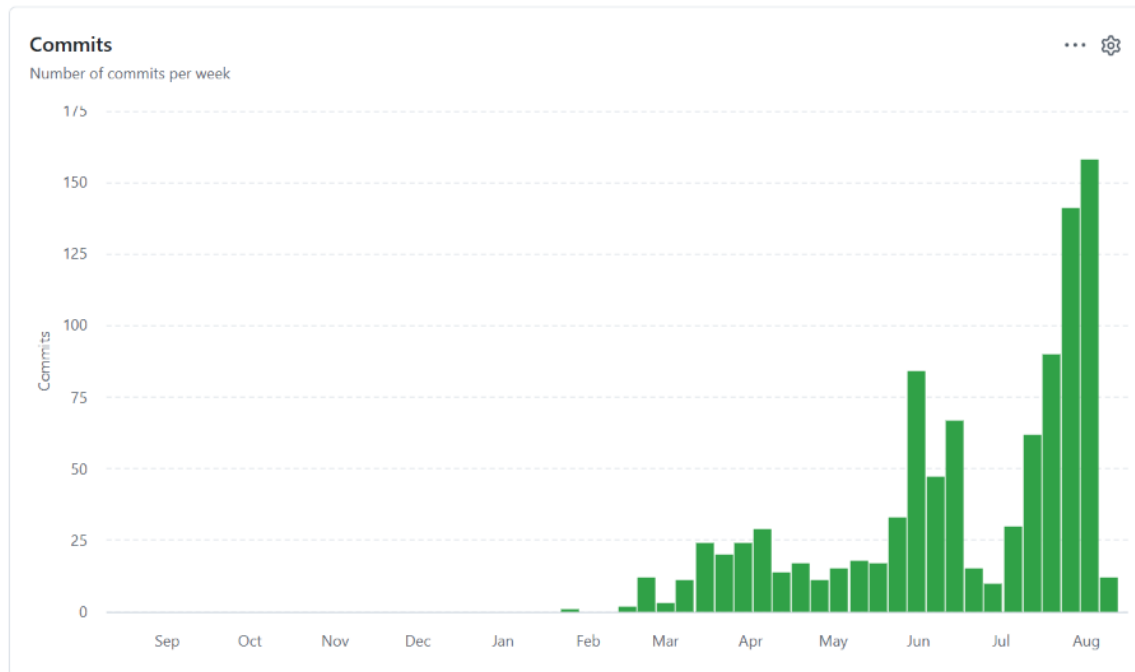
BuildStorm 与 Cagent 测例全部通过，性能得分可观。



八个月， 从零设计、构建完整内核

- 内核代码量：156096 行
- 工程时间：8 个月
- Commit 数目：1242

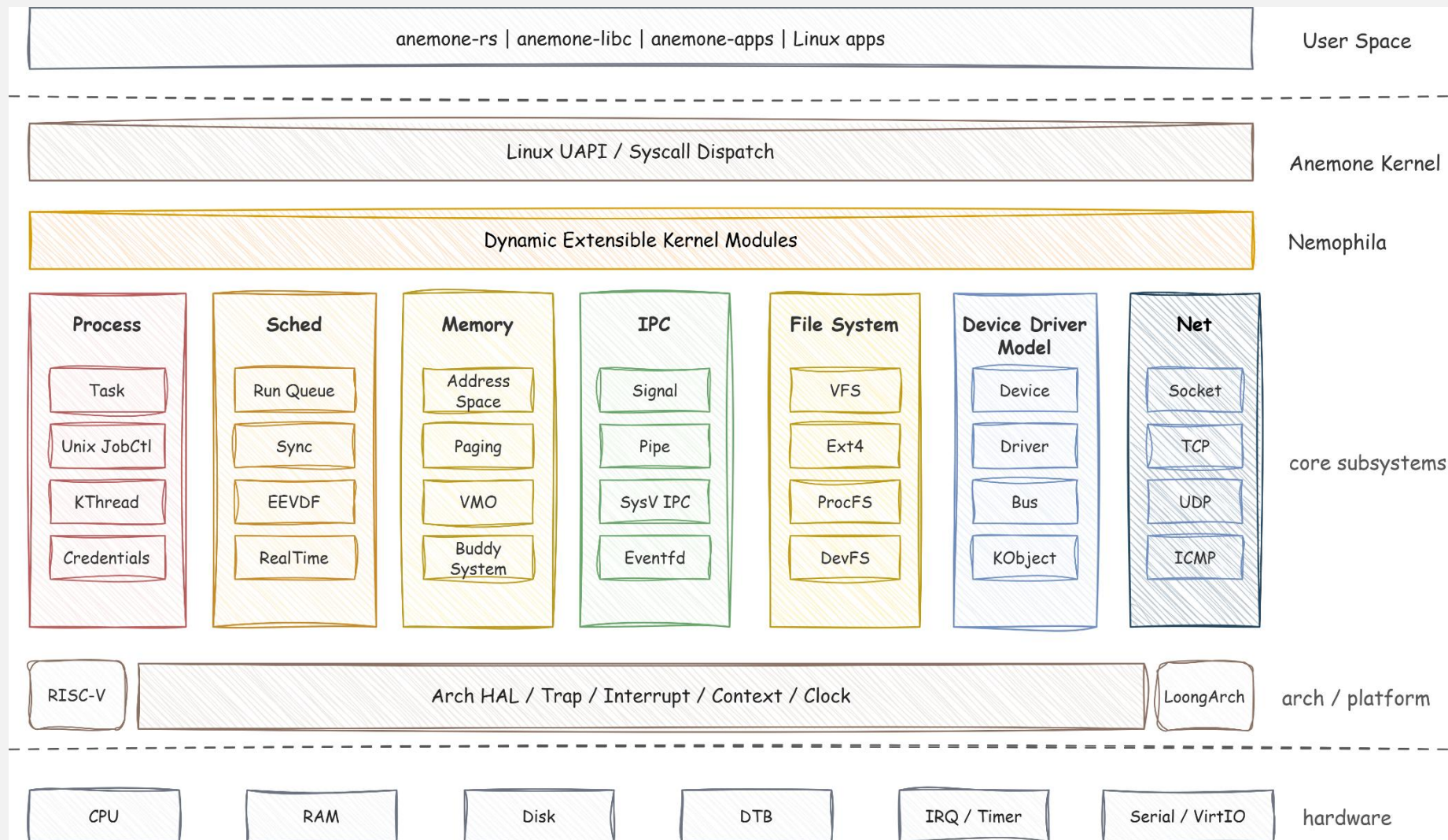
Commits over the last year of anemone-os/anemone



项目	Rust 有效代码行	Rust 物理行数
anemone-rs	3,471	4,144
anemone-abi	3,871	4,776
anemone-kernel	148,754	182,121
合计	156,096	191,041



➤ Anemone 的整体架构



➤ Anemone 核心能力全景

核心能力	实现功能
进程与调度	类LWP设计; SMP; Fair, Realtime, 多调度类动态切换; Unix作业控制
动态扩展	沙箱化内核模块; 热插拔; WebAssembly; 同一模块制品可同时运行在不同架构; Rust SDK
内存管理	Zircon VMO架构; 写时复制; 异常表驱动的用户访存; 匿名零页共享; IO Remap
VFS	多重挂载 / Bind Mount; ICache / Page Cache / DCache; Direct IO; ProcFs; DevFs; SysFs
设备驱动模型	Bus + Driver + Device模型; 支持中断IO与DMA IO; PCI、VirtIO、MMC、GMAC; 多层次IRQ; 设备树/EFI解析
网络	统一Socket前端; TCP; UDP; ICMP; NetLink; IPv4; 本地回环
硬件抽象层	RISC-V64 / LoongArch64; 架构特有能力的旁路
构建系统	Xtask; KernelConfig; RootFsConfig; SystemTarget; App/Module 完整可配置自动化构建链路



➤ 目前已经支持的应用程序举例

- OpenJDK25 HotSpot JIT, 运行 Minecraft Java 26.2 Server
- Cargo / RustC 足以编译大型代码库, 例如 arceos-helloworld
- Apt 已经支持直接使用apt install大量包
- OpenSSH/Dropbear 已经可以建立sshd和通过ssh连接其他主机
- Golang 支持CGO与原生GO大规模编译
- Nginx 可以直接启动服务器, 并通过网络访问
- Qemu-System 支持系统级别模拟。可以嵌套启动Anemone。
-



➤ 一次MC Server启动的日志

```
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by org.joml.MemUtil$MemUtilUnsafe (file:/usr/games/libraries/org/joml/joml.jar)
WARNING: Please consider reporting this to the maintainers of class org.joml.MemUtil$MemUtilUnsafe
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
[20:23:11] [ServerMain/INFO]: Environment: Environment[sessionHost=https://sessionserver.mojang.com, servicesHost=https://api.minecraftservices.com]
[20:23:31] [Worker-Main-1/INFO]: No existing world data, creating new world
[20:23:55] [ServerMain/INFO]: Loaded 1585 recipes
[20:23:55] [ServerMain/INFO]: Loaded 1688 advancements
[20:23:58] [Server thread/INFO]: Starting minecraft server version 26.2
[20:23:58] [Server thread/INFO]: Loading properties
[20:23:58] [Server thread/INFO]: Default game type: SURVIVAL
[20:23:58] [Server thread/INFO]: Generating keypair
[20:24:00] [Server thread/INFO]: Starting Minecraft server on *:25565
[20:24:01] [Server thread/WARN]: Failed to find a usable hardware address from the network interfaces; using random bytes: ae:7e:3f:7
[20:24:03] [Server thread/INFO]: Preparing level "world"
[20:24:12] [Server thread/INFO]: Selecting global world spawn...
[20:25:14] [Server thread/INFO]: Loading 0 persistent chunks...
[20:25:14] [Server thread/INFO]: Preparing spawn area: 100%
[20:25:14] [Server thread/INFO]: Time elapsed: 61950 ms
[20:25:14] [Server thread/INFO]: Done (70.907s)! For help, type "help"
[20:25:14] [Server thread/INFO]: Saving chunks for level 'ServerLevel[world]'/minecraft:overworld
[20:25:15] [Server thread/INFO]: Saving chunks for level 'ServerLevel[world]'/minecraft:the_nether
[20:25:15] [Server thread/INFO]: Saving chunks for level 'ServerLevel[world]'/minecraft:the_end
[20:25:15] [Server thread/INFO]: ThreadedAnvilChunkStorage (world): All chunks are saved
[20:25:15] [Server thread/INFO]: ThreadedAnvilChunkStorage (DIM-1): All chunks are saved
[20:25:15] [Server thread/INFO]: ThreadedAnvilChunkStorage (DIM1): All chunks are saved
[20:25:15] [Server thread/INFO]: ThreadedAnvilChunkStorage: All dimensions are saved
help
[20:25:23] [Server thread/INFO]: /advancement (grant|revoke)
[20:25:23] [Server thread/INFO]: /attribute <target> <attribute> (get|base|modifier)
[20:25:23] [Server thread/INFO]: /execute (run|if|unless|as|at|store|positioned|rotated|facing|align|anchored|in|summon|on)
[20:25:23] [Server thread/INFO]: /bossbar (add|remove|list|set|get)
[20:25:23] [Server thread/INFO]: /clear [<targets>]
[20:25:23] [Server thread/INFO]: /clone (<begin>|from)
[20:25:23] [Server thread/INFO]: /damage <target> <amount> [<damageType>]
```



➤ Anemone 开发历程





设计主线

边界 · 所有权 · 兼容



➤ 为什么坚持从零设计

- 兼容对外协议

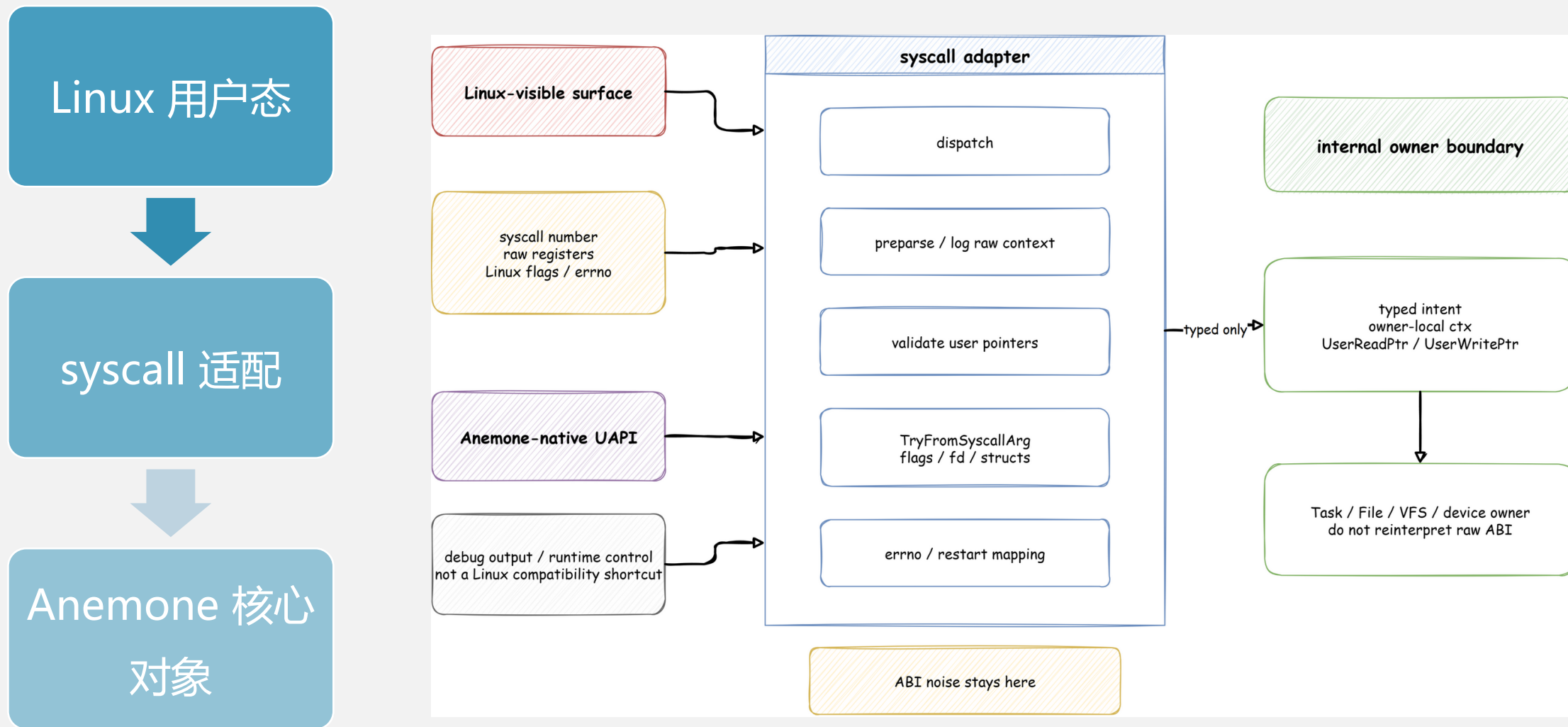
Linux ABI 兼容停留在 syscall / UAPI 边界；不复制 Linux 内部对象与历史实现结构

- 重建内部对象模型

核心对象围绕 Rust 所有权与单一真相源设计；调试、审查和演进路径更清楚



➤ 兼容逻辑停留在边界层





核心机制

机制 · 策略 · 创新点

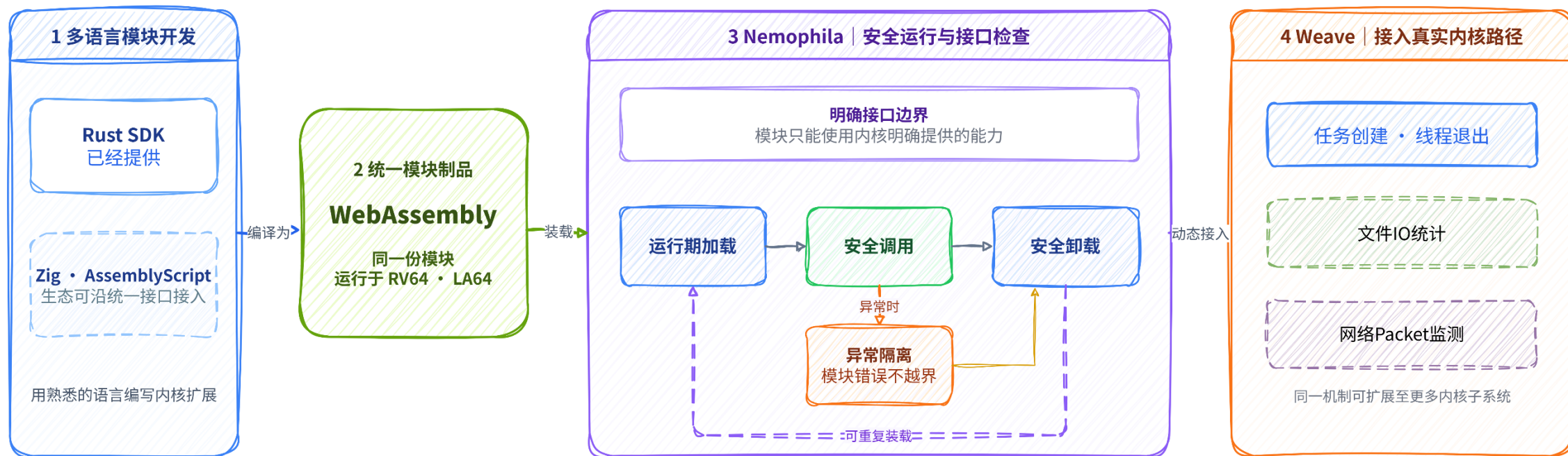


➤ Nemophila – 基于WASM的内核动态扩展机制

模块热插拔；WASM天然提供沙箱性质；一次编译，多架构装载；内核Hook

Nemophila：让内核在运行期安全扩展

我们将多语言工具链、统一 Wasm 制品与内核观测点连接成一条完整的动态扩展路径



统一制品 · 受约束接口 · 完整生命周期 · 可扩展观测面

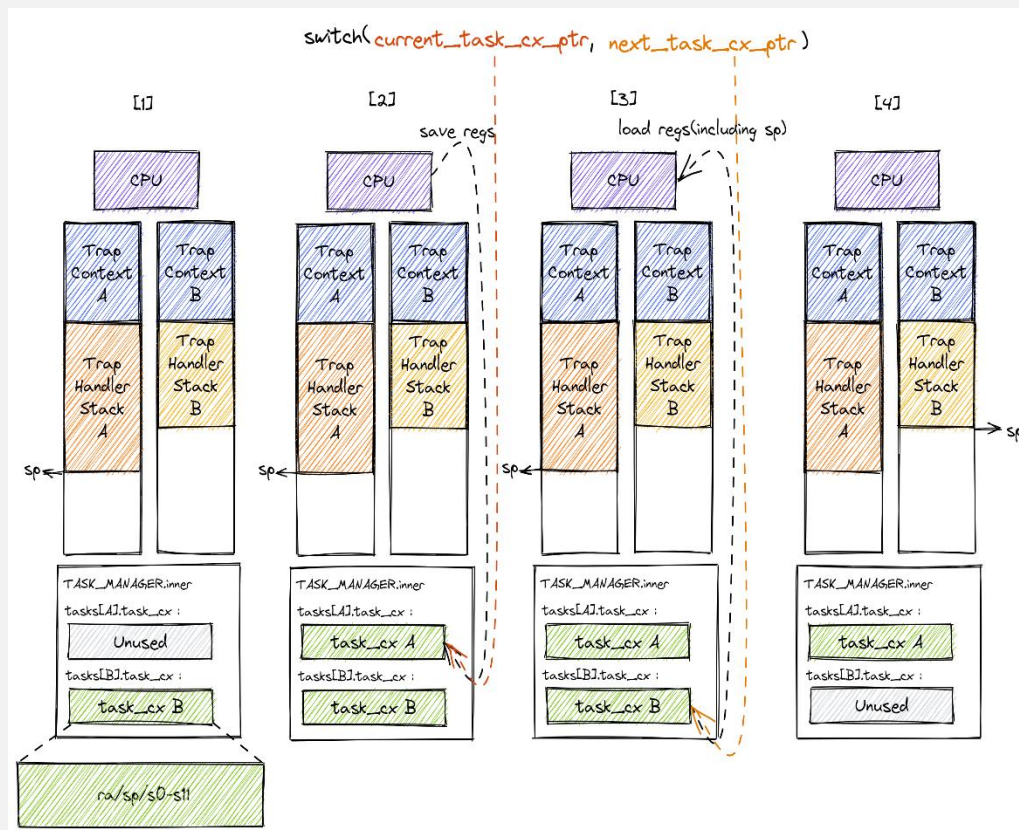
➤ 动态多调度策略

多种同步原语:

- Event 等待与发布
- Latch 一次性的等待闸门
- OneShot 一次性的数据通道

动态调节调度策略:

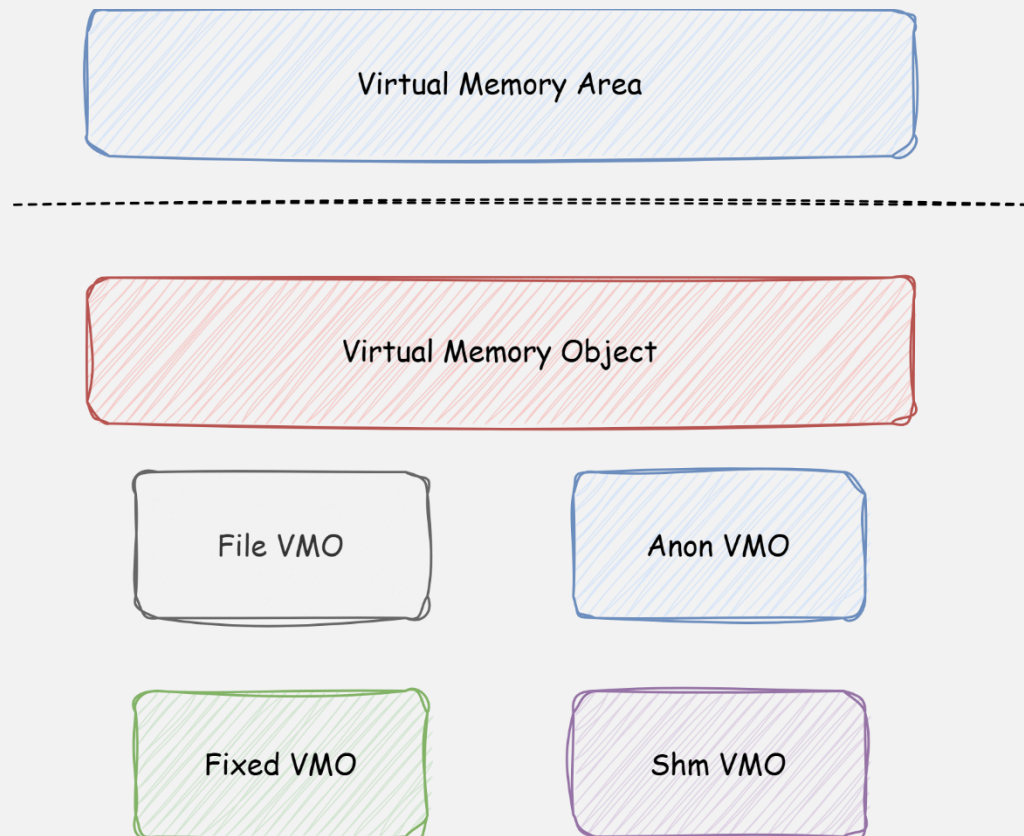
- 公平调度类: **EEVDF**; Stride
- 实时调度类: 轮转算法; FIFO
- Idle调度类





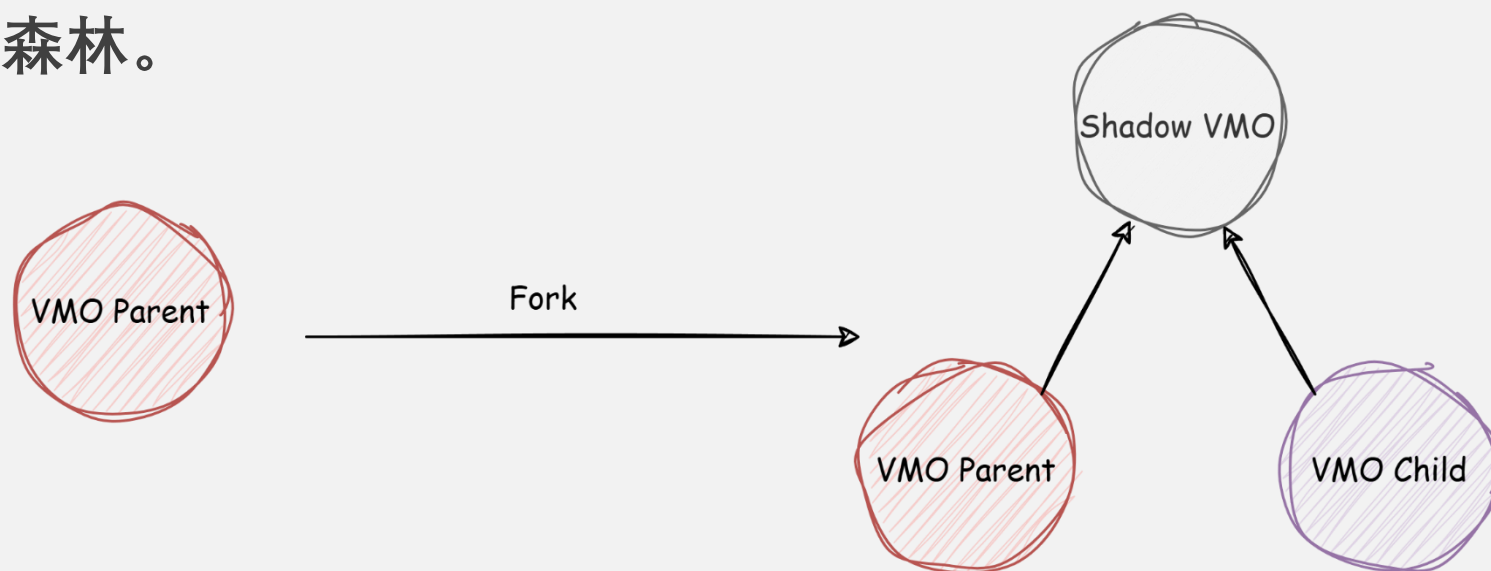
➤ VMO 统一内存对象：脱胎于Zircon

- 微内核的思想，适配，改进，应用于宏内核
- 匿名页、文件映射、共享内存与COW 由 VMO / backing object 组织
- 不同地址空间有不同VMA
- VMA存在对应的VMO后端支持
- 一个VMO可以支持多个地址空间的不同VMA



➤ Fork 写时复制 · Shadow VMO设计

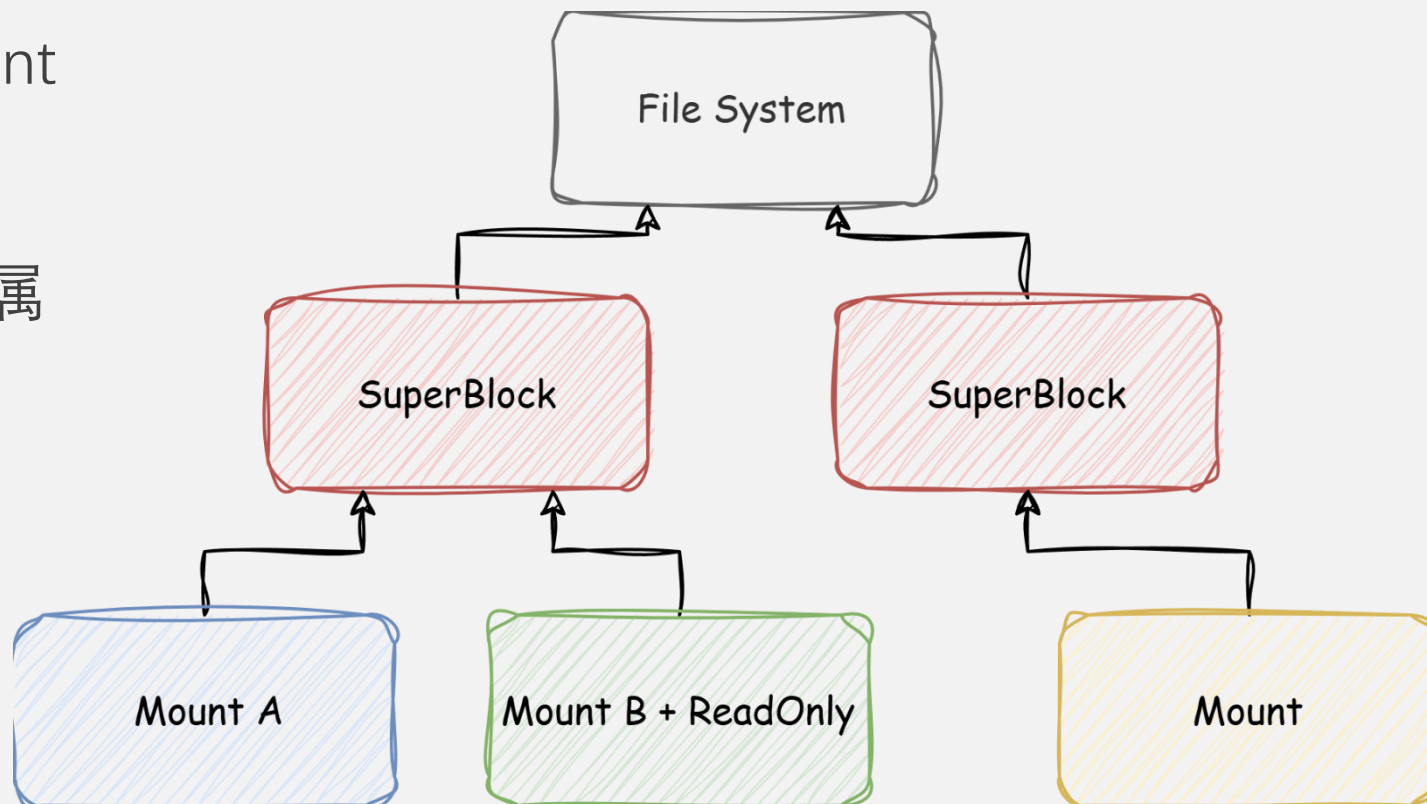
- COW 延迟复制·匿名 VMO 可共享全局只读零页
- Fork后，一个VMO转变为三个VMO——COW。
- Shadow VMO将形成树，整个内核所管理的地址空间将是一片森林。





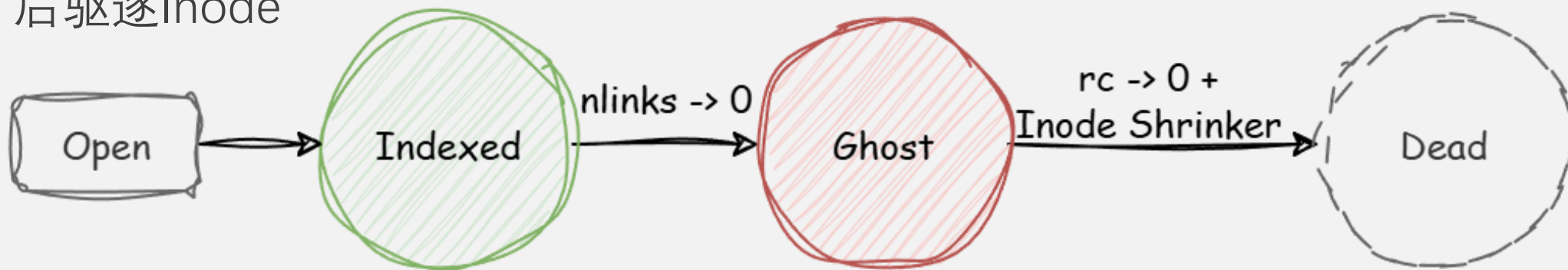
➤ VFS 对象模型

- filesystem, superblock, inode, dent ry, file 与 path view 分层
- 一个SuperBlock可以支持多个属性不同的Mount



➤ VFS 缓存

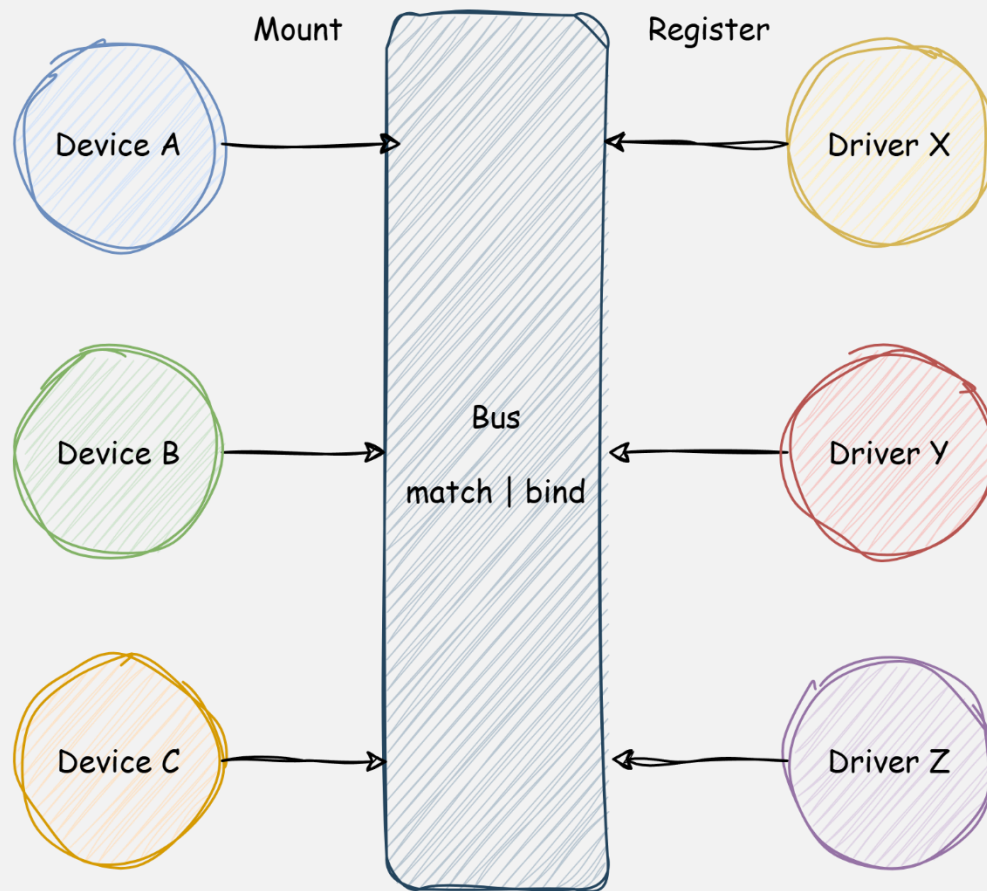
- ICache 稳定 inode 身份
- INode的生命周期
 - Alive: Inode被索引/Indexed, 可以被通过Ino查找
 - Ghost: 最后一个硬链接被删除后, Inode被去索引/Unindexed, 在文件树中不再可见; 已有引用仍可继续使用
 - Evicted: 引用计数归零后, VFS将数据同步回块设备, 最后驱逐Inode



- 通过Page Cache, DEntry Cache**提高性能**。

➤ Bus + Device + Driver: 设备驱动模型

- 设备和驱动被挂载到总线上, 相互发生匹配
- 支持Platform, VirtIO, PCI, MMC等多种类型总线
- 实现完整的PCIe ECAM枚举流程, 支持多级枚举和动态BAR分配。
- 支持多层次总线





➤ VisionFive2 和 2K1000 星光板硬件完整适配

- 为VisionFive2星光板制作了完整适配方案。

- 支持基于LoongArch64的系统。

- 完整GMA2D和AHB-APB在物理设计。

- LoongArch64平台架构。

← → ↺ 192.168.2.3

🔍 ☆

品

Anemone Alpine HTTP service

HTTP and HTTPS are available.

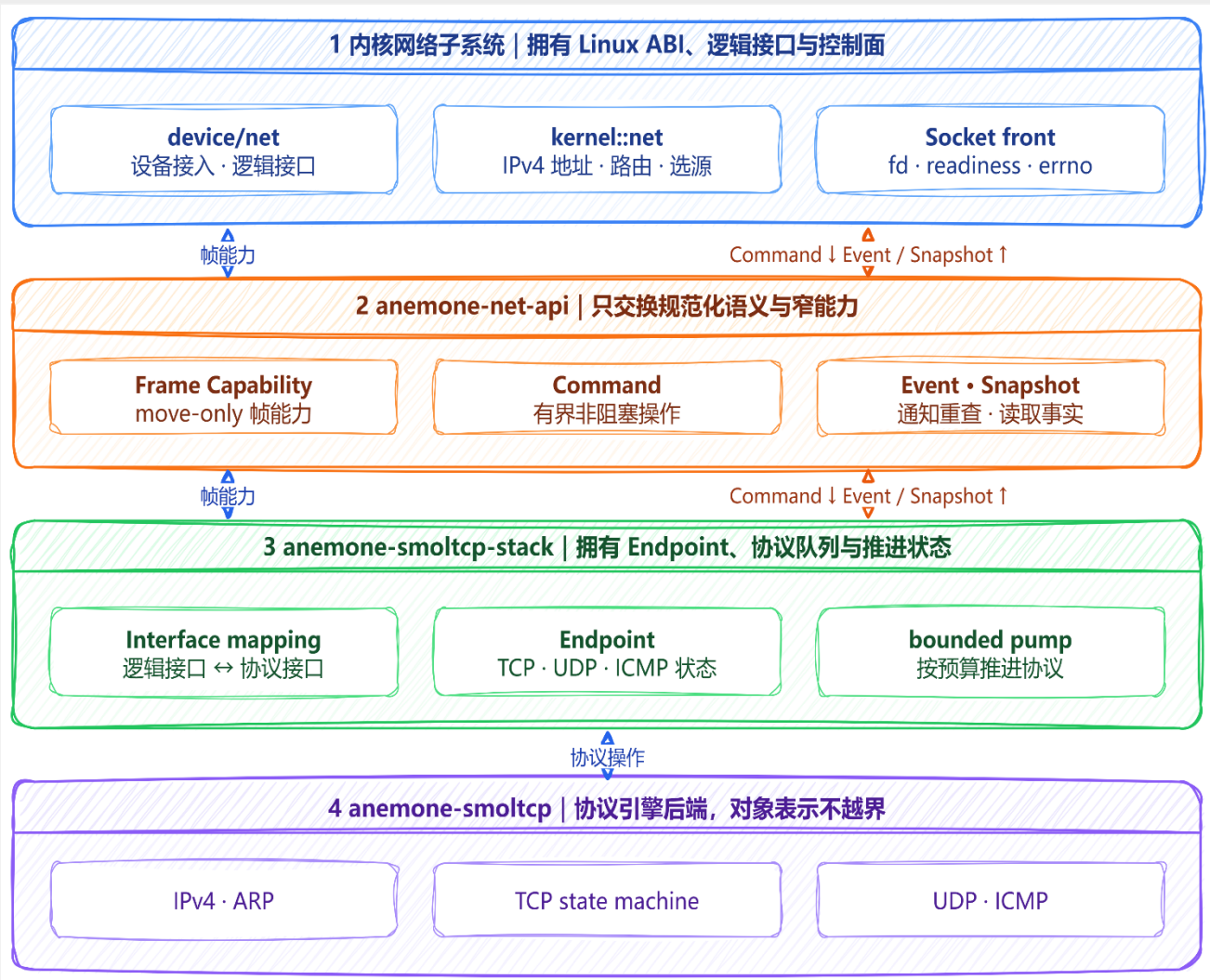
46 2026

```
root@anemone:~# rc-service nginx start
* Validating provisioned Anemone machine identity ... [ ok ]
* Starting nginx ... [ ok ]
root@anemone:~#
```




➤ 四层网络架构

- 内核 Socket、Net API、协议栈 owner 与 smoltcp 后端各自拥有状态
- 内核处理Linux语义，阻塞与同步，Socket状态维护
- Net-API 负责语义和能力的确认，可以理解为网络栈的抽象层，屏蔽后端语义，允许此后扩展和切换
- Smoltcp-stack是smoltcp的后端适配。





➤ Agentic Coding workflow

- 初赛前三个月，我们完全古法编程
- 初赛的最后一个月，为了追赶进度，我们开始逐步使用LLM辅助开发。
- 到了决赛，我们大量使用AI辅助开发。

AI 生成内容本身不等于低质量；真正决定质量的是我们是否能给出清楚的上下文、是否能识别错误方向、是否能将输出放回源码和测试中验证。以及，是否能建立完备的Harness。在这过程中，开发者始终把握架构设计与大方向。

➤ 三种 workflow

- Patch——最小化的代码迭代，不引入新的系统不变量变更
- 小迭代——引入或改变系统不变量，但处于一个子系统内部
- RFC——系统不变量的新增或更改，且横跨多个子系统



➤ 创新

- 如前所述，内核设计上，我们没有参考往届队伍的架构，完全从零设计。
- 相较于工业化的内核（Linux/Zircon），我们没有完全照搬，而是博采众长，对宏内核和微内核的优点兼收并蓄。
- 设计实现了Nemophila，基于沙箱化WebAssembly的内核扩展机制。
- 支持调度策略的动态切换；完备的设备驱动模型；独立设计的硬件抽象层；分散式的Syscall注册机制，等等。

➤ 参考

- Linux 6.6。Linux UAPI、进程与信号语义、VFS、设备驱动、网络、调度与各架构实现的主要参考。
- Zircon。VMO、地址空间 backing object 与写时复制设计。
- Lwext4。Ext4实现。
- SmoITCP。我们的网络子系统的协议栈后端。



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

感谢聆听